

Datacenter Design and Operation

Synthetic and Reworked Notes

Edited by Fabio Piscitelli M.715339

Academic Year 2025/2026

Contents

1	Introduction, Architecture and Infrastructure	1
1.1	Evolution: From Server Rooms to AI Factories	1
1.1.1	From Mainframes to the Dot-Com Bubble	2
1.1.2	The Era of Artificial Intelligence and the Top500	2
1.2	Fundamentals and Physical Constraints	2
1.2.1	Floating Floor	3
1.3	Cooling and Thermal Management	3
1.3.1	Air Architecture: CRAC Systems and Flows	4
1.3.2	Liquid Cooling	5
1.4	Energy Efficiency (PUE)	5
1.5	Security and Continuity	5
2	Cabling, Switching and Network Fabric	7
2.1	The Physical Medium: Copper and Optical Fiber	7
2.1.1	Copper Cables and Ethernet	7
2.1.2	Optical Fiber and Transceivers	8
2.2	Internal Switch Architecture	9
2.2.1	Software-Defined Networking (SDN) and SONiC	10
2.3	Network Topologies: From 3-Tier to Spine-and-Leaf	13
2.4	Network Segmentation and Virtualization	15
2.4.1	Jumbo Frames and MTU	15
2.4.2	VLAN (Virtual LAN)	15
2.4.3	VXLAN and Overlay	15
2.5	High-Performance Fabric (HPC and AI)	16
3	Storage: Architectures and Services	17
3.1	Evolution of the Storage Medium	17
3.1.1	From Magnetic Tape to HDD and NVMe SSD	17
3.1.2	NAND Cells and Asymmetry	18
3.1.3	Performance Metrics: IOPS, Latency and Bandwidth	18
3.2	Aggregation and Protection: RAID and Resilience	19
3.3	Storage Architectures in the Datacenter	19
3.3.1	SAN (Storage Area Network)	19
3.3.2	NAS (Network Attached Storage)	20
3.3.3	Object Storage	20
3.3.4	HCI (Hyper-Converged Infrastructure)	21
3.3.5	Storage for AI: Vector Database	21
3.4	Advanced Services and Resilience	21

3.4.1	Efficiency: Deduplication, Compression and Tiering	21
3.4.2	Snapshot, Thin Provisioning and Replication	22
3.4.3	Backup vs Replication	22
4	Compute and Virtualization	24
4.1	Compute Architecture: CPU, GPU and NPU	24
4.1.1	Energy Sustainability	24
4.1.2	CPU Architecture: Pipeline and Multi-Core	24
4.1.3	GPU and NPU: The Paradigm Shift	26
4.2	Physical Server Architectures and BMC	26
4.2.1	Rack Servers (1U / 2U)	27
4.2.2	Blade Servers	28
4.2.3	HGX Servers (AI GPU Server)	29
4.2.4	Hyperscaler Standardization: Open Compute Project (OCP)	29
4.3	Virtualization and Live Migration	30
4.3.1	Hypervisor Evolution: From CPU Rings to Hardware Virtualization	30
4.3.2	Hardware Abstraction: vCPU, Pause and Resume	31
4.3.3	The Virtual Switch (vSwitch)	31
4.3.4	Live Migration	31
4.4	Containerization	32
4.4.1	Isolation Mechanisms and Vulnerabilities	32
4.4.2	Docker Architectural Details	32
4.4.3	Network: VM vs Container	33
5	Cloud Computing, Orchestration and Security	34
5.1	The Cloud Computing Model	34
5.1.1	Definition and Characteristics (NIST)	35
5.1.2	Service Models	35
5.2	Reference Architecture (Reference Model)	36
5.2.1	Cloud Service Lifecycle	36
5.3	Business Continuity and Data Protection	37
5.3.1	Resilience Techniques and Geographical Limits	37
5.3.2	Application Resiliency: Design for Failure	38
5.4	Security	38
5.4.1	Classic Threats and Attacks	39
5.4.2	Identity and Policy Management (AAA and RBAC)	39
5.4.3	Defense-in-Depth and Zero Trust Approach	39
5.4.4	Risk Management (GRC)	40
6	Operations and Final Considerations	41
6.1	Service Operations Management	41
6.1.1	Monitoring and Predictive Failure	41
6.1.2	Capacity Monitoring and Management	42
6.2	Change and Failure Management	42
6.2.1	Change Management	42
6.2.2	Incident vs. Problem Management	42
6.3	Final Considerations on the Datacenter	43

Chapter 1

Introduction, Architecture and Infrastructure

1.1 Evolution: From Server Rooms to AI Factories

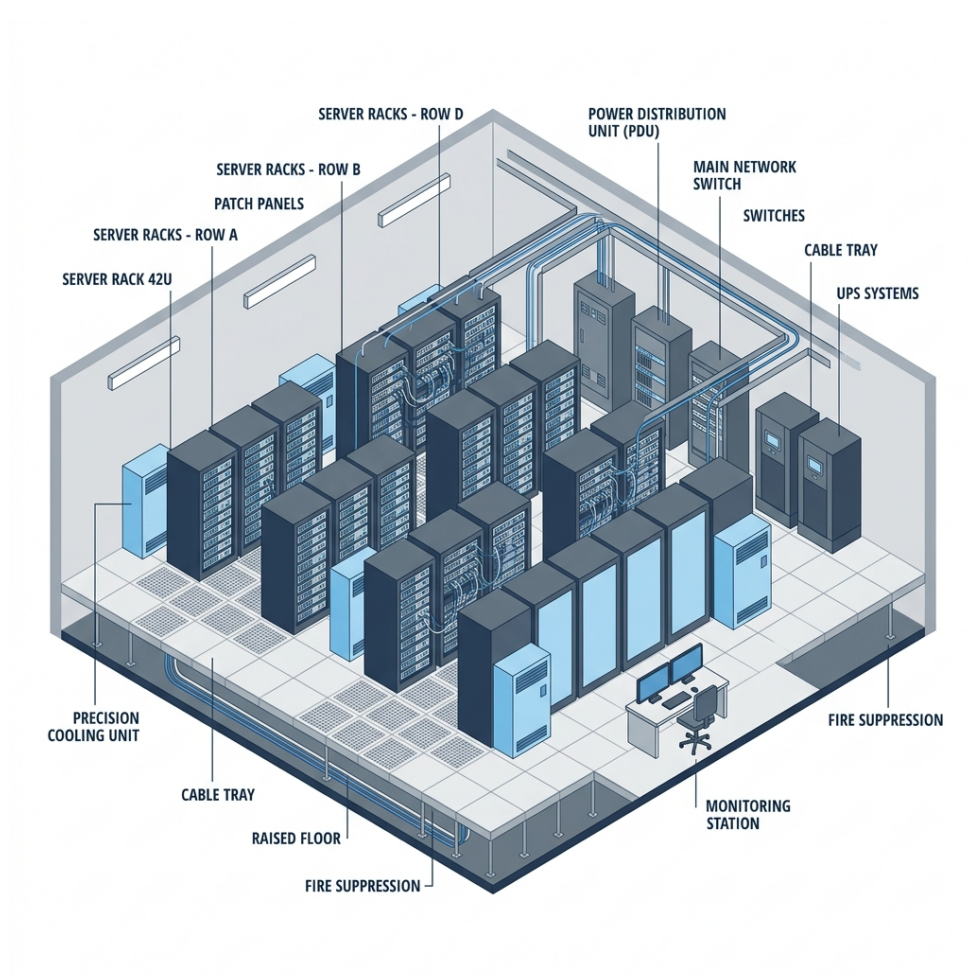


Figure 1.1: Isometric illustration of a modern high-density datacenter.

1.1.1 From Mainframes to the Dot-Com Bubble

Historically, centralized computing started in the 60s and 70s with **Mainframes** (e.g. IBM systems): gigantic cabinets enclosed in sterile rooms ("glass houses") which users accessed via dumb terminals (green screens). In this phase, the system was a monolithic, proprietary, and highly centralized block. In the 80s and 90s, there was a shift to the **Client-Server** model, with x86 servers (open architecture) distributed in simple corporate "server rooms", often repurposed closets with a commercial air conditioner and no real engineering design. At the end of the 90s, the explosion of the Internet led to the **Dot-Com bubble**: the need to scale globally pushed companies to rent space (Colocation) in gigantic specialized buildings managed by telcos and providers, to ensure 24/7 accessibility. This gave birth to the true modern concept of a Datacenter.

1.1.2 The Era of Artificial Intelligence and the Top500

Today, these are critical infrastructures at a national level, requiring investments of hundreds of millions, if not billions, of euros. The explosion of artificial intelligence (AI) has led to the era of **AI Factories**: industrial complexes capable of absorbing up to 1-2 Gigawatts, the equivalent of an entire city. The power of these clusters is constantly tracked by two biannual global rankings (started in 1993):

- **Top500**: Measures absolute performance (linear algebra benchmarks in PetaFLOPS).
- **Green500**: Measures energy efficiency (GigaFLOPS per Watt), highlighting the profound **environmental impact** of these structures.

AI has radically altered past paradigms: while traditional Cloud computing uses virtualization to maximize the use of hardware (normally idle for 90% of the time), AI is the first *workload* that **fully saturates the physical hardware**. This makes virtualization useless in the AI stack and dictates the use of hardware architectures distributed across tens of thousands of interconnected GPUs.

1.2 Fundamentals and Physical Constraints

The construction of a modern datacenter clashes with engineering limits defined as **Hard Thresholds**: physical thresholds beyond which it is not simply possible to "scale" (enlarge) a technology, but a radical rethinking becomes necessary.

Attention: Scale Up vs Scale Out

- **Scale Out**: adding new nodes or servers to distribute the load.
- **Scale Up**: increasing the capacity of the single node (or, in the electrical case, of the single cable or building).

The electrical infrastructure of an entire datacenter is forced to operate in a **Scale Up** logic: as consumption increases, increasingly thick and expensive copper cables are required.

1.2.1 Floating Floor



Figure 1.2: Lifting of a tile from the raised floor in a data center (Wikimedia Commons).

One of the most widespread engineering legacies is the **Floating Floor** (raised floor). Consisting of panels raised on metal pedestals, in modern DCs it can support up to 1.5 tons per square meter. The two main advantages are:

1. **Cable management:** orderly routing of cabling infrastructure and plumbing.
2. **Air plenum:** creation of a pressurized duct under the floor to distribute cold air from the cooling system towards the grilles positioned in front of the racks.

1.3 Cooling and Thermal Management

Due to the extreme heat density, caused by the high **TDP (Thermal Design Power)** of new processors (reaching a Kilowatt per single GPU), thermal control has become the most critical aspect of the design. Extreme temperatures can melt silicon components; the interruption of the cooling system, combined with the attempt of the server fans to push air, causes exponential thermal increases that can damage the entire room in a few minutes.

1.3.1 Air Architecture: CRAC Systems and Flows

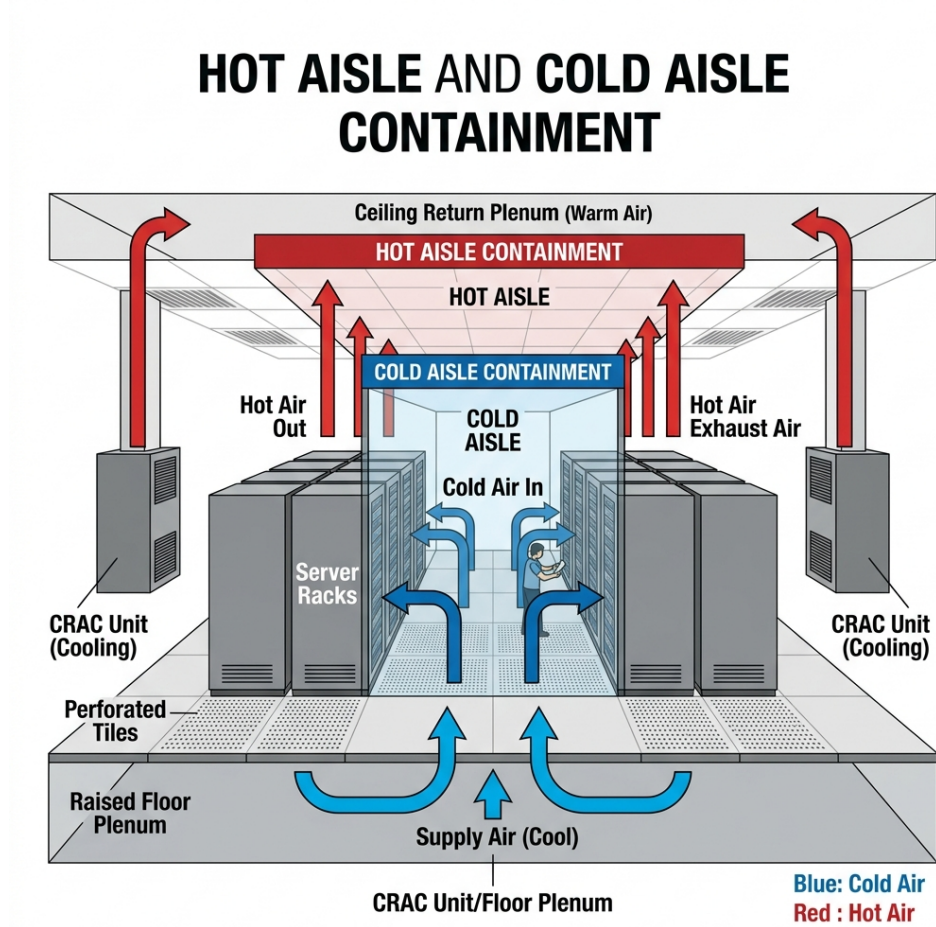


Figure 1.3: Airflow diagram: Cold Aisle Containment and Hot Aisle.

CRAC (Computer Room Air Conditioning) systems rely on the raised floor and the principle of strict separation between the *Cold Aisle* and the *Hot Aisle*. The airflow follows a precise path: the CRAC units pump chilled pressurized air into the *plenum* (the space under the raised floor). The air flows out of the perforated grates exactly in front of the racks (Cold Aisle). The server fans suck the air in from the front, cool the internal heatsinks, and expel the blistering hot air out the back into the Hot Aisle, where it rises to the ceiling to return to the CRACs and be cooled again using water heat exchangers (Chillers).

Golden Rule

Never mix hot air and cold air. If the flows mix, efficiency plummets and CPUs risk *thermal throttling*. For this reason, glass doors and closed roofs are used to create systems of **physical containment** for the aisles.

However, air is a terrible thermal conductor. For high loads (Racks beyond 15-20 kW, not to mention modern 200 kW AI racks), CRAC air systems are insufficient.

1.3.2 Liquid Cooling

Liquid cooling is today a de facto standard for high performance computing (HPC):

- **Direct-to-Chip:** water-cooled copper plates placed directly in contact with CPU, GPU and RAM. It removes the need for high-speed fans.
- **Waterless Cooling:** use of dielectric fluids (e.g. glycol) in piping. In case of a spill, they do not cause short circuits.
- **Immersion Cooling:** complete immersion of the motherboard in mineral oil. Eliminates any localized heat point (hot-spot), but makes maintenance complex and expensive.

1.4 Energy Efficiency (PUE)

The efficiency of a datacenter is measured with a standard dimensionless value, the **PUE (Power Usage Effectiveness)**:

$$PUE = \frac{\text{Total Facility Power}}{\text{IT Equipment Power}}$$

The closer the PUE is to 1, the higher the efficiency (it means energy is not being wasted on cooling). The best modern data centers have an annual PUE around 1.15 - 1.20.

The Pitfalls of PUE

Although PUE is the de facto standard, it presents several critical ambiguities that can be misleading:

- **Impact of Seasonality:** PUE is not a fixed value. A datacenter that exploits outside air cooling (Free Cooling) will measure an excellent PUE (e.g. 1.05) in winter, but will see efficiency plummet (e.g. 1.50) in summer due to the forced activation of mechanical compressors.
- **The Empty Datacenter Paradox:** A newly opened data center with very few servers turned on (very low IT Power) will paradoxically have an abysmal PUE. This is because industrial cooling systems (chillers, pumps) have an unavoidable baseline consumption even when running at minimum, skewing the mathematical fraction.
- **Economic Evaluation (Domestic Splits):** To cool a small server room, installing an industrial plant (PUE 1.3) would cost tens of thousands of euros and require expensive maintenance contracts. Simple "split" domestic air conditioners are often chosen: they bring the PUE to a disastrous 2.0, but they are economically the most sensible choice in terms of initial investment and practical reliability.

1.5 Security and Continuity

The logical design follows the so-called **CIA Triad**:

- Confidentiality
- Integrity
- Availability

Electrical supply is vital. The distribution follows a rigorous chain: From the external alternating current (AC) grid, it passes to industrial **UPS (Uninterruptible Power Supply)** (based on huge batteries and pre-compressed spring systems for instantaneous switching) which stabilize the current and prevent micro-interruptions. From the UPS it goes to the hierarchical **PDU (Power Distribution Unit)**, up to the servers which internally convert AC to direct current (DC).

The need for extreme resilience clashes with practical and geographical limits. The location of data centers must in fact take into account critical variables such as seismic risk, temperature ranges, immediate access to the ultra-high voltage transmission grid and regulatory obligations imposed by GDPR.

Chapter 2

Cabling, Switching and Network Fabric

2.1 The Physical Medium: Copper and Optical Fiber

The network is the vascular system of the datacenter. The evolution of traffic density has marked the progressive abandonment of copper in favor of optical fiber.

2.1.1 Copper Cables and Ethernet

Historically, computer networks faced the problem of collisions on shared cables. A widespread topology was the **Token Ring (IEEE 802.5)**, a physical ring network in which an electronic "token" was passed between computers: only those holding the token could transmit. Although it guaranteed predictable latency, the failure of a single cable or node paralyzed the entire ring. Ethernet won the commercial battle thanks to the star topology and the CSMA/CD protocol, much more robust and flexible but prone to inefficiencies and collisions at high loads before the introduction of switching.

Regarding cabling, data centers relied heavily on copper (e.g. the old **Cat 5** or Cat 5e cables which limited speeds to 100 Mbps or 1 Gbps). The physical limits of copper (crosstalk and signal attenuation) and the need to manage a precise *pin-out* for contact alignment made evolution increasingly complex. Today, the traditional copper cable (Ethernet standard with RJ45 connector) is confined to the practical limit of **10 Gbps (Cat 6A)**. Going further with standards like Cat 7 or Cat 8 is impractical: the cables become thick, rigid, and difficult to manage (*cable management* is vital, as messy cabling blocks airflow and destroys the thermal efficiency of the rack).

2.1.2 Optical Fiber and Transceivers



Figure 2.1: Optical fiber connectors (LC and SC compared).

Optical transmission eliminates thermal dissipation and dramatically increases bandwidth, but requires the use of **transceivers** (hot-swappable modules) to convert the electrical signal into an optical one. The evolution of these modules has defined datacenter speeds:

- **SFP28:** Standard for 25 Gbps server ports (single lane).
- **QSFP Family (Quad SFP):** Uses 4 internal lanes. Includes QSFP+ (40G), QSFP28 (100G, standard for uplinks), up to the modern **QSFP56** (200G) and **QSFP-DD** (up to 800G).

The performance leap beyond 100G did not happen by adding cables, but by changing the signal modulation from **NRZ** (1 bit per symbol) to **PAM4** (2 bit per symbol, doubling the bandwidth at the same frequency). Modules use standardized color codes engraved on the body (e.g., blue for single-mode, black/gray for multi-mode).

Fibers are divided into two main families:

- **Multi-mode (OM):** wider core (50 μm), allows more photon bounces. It has short operating distances (up to $\sim 100\text{-}300$ m) but lower costs. It is the dominant standard *inside* the datacenter.
- **Single-mode (OS):** very thin core (9 μm), allows a single straight path. Ideal for long-range metropolitan links (up to 40 km).

2.2 Internal Switch Architecture

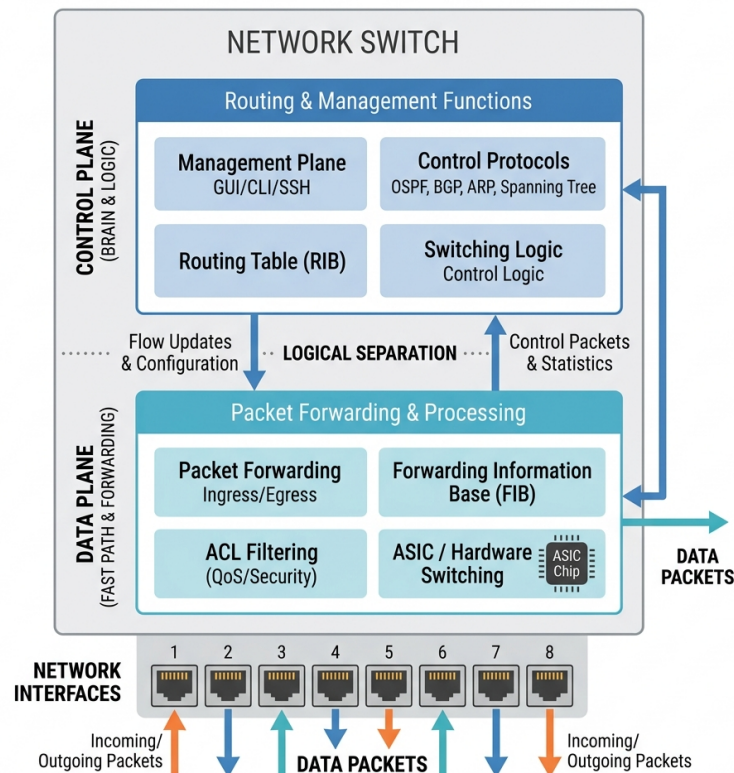


Figure 2.2: Logical architecture: separation between Control Plane and Data Plane in a network device.

In the past, the dominant network architecture was based on huge proprietary *modular chassis* (e.g. large Cisco core switches), expensive and difficult to scale gradually. The modern switch, driven by the hyperscale needs of the cloud, has abandoned chassis in favor of the **Fixed Form Factor** (1U or 2U), often called *White-box switch*.

2.2.1 Software-Defined Networking (SDN) and SONiC

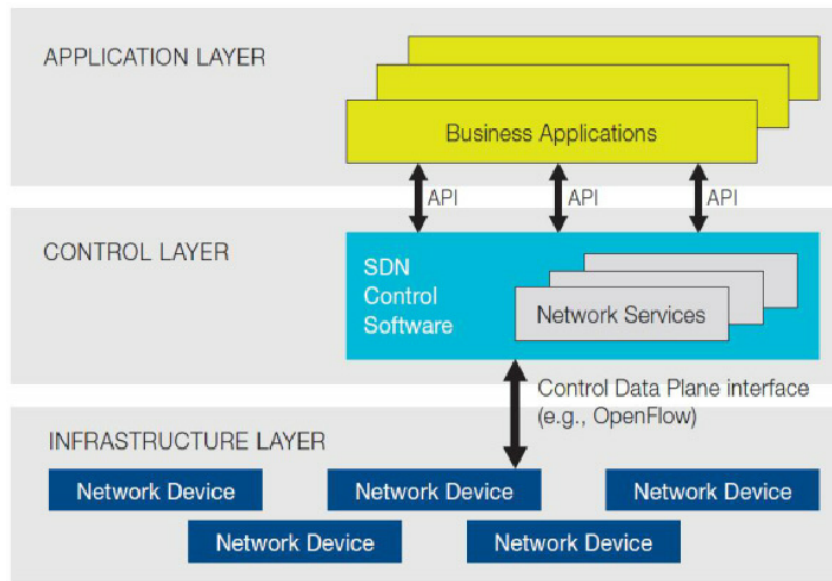


Figure 2.3: SDN Model: network intelligence is abstracted from the underlying physical infrastructure.

The great revolution was the conceptual separation into two planes:

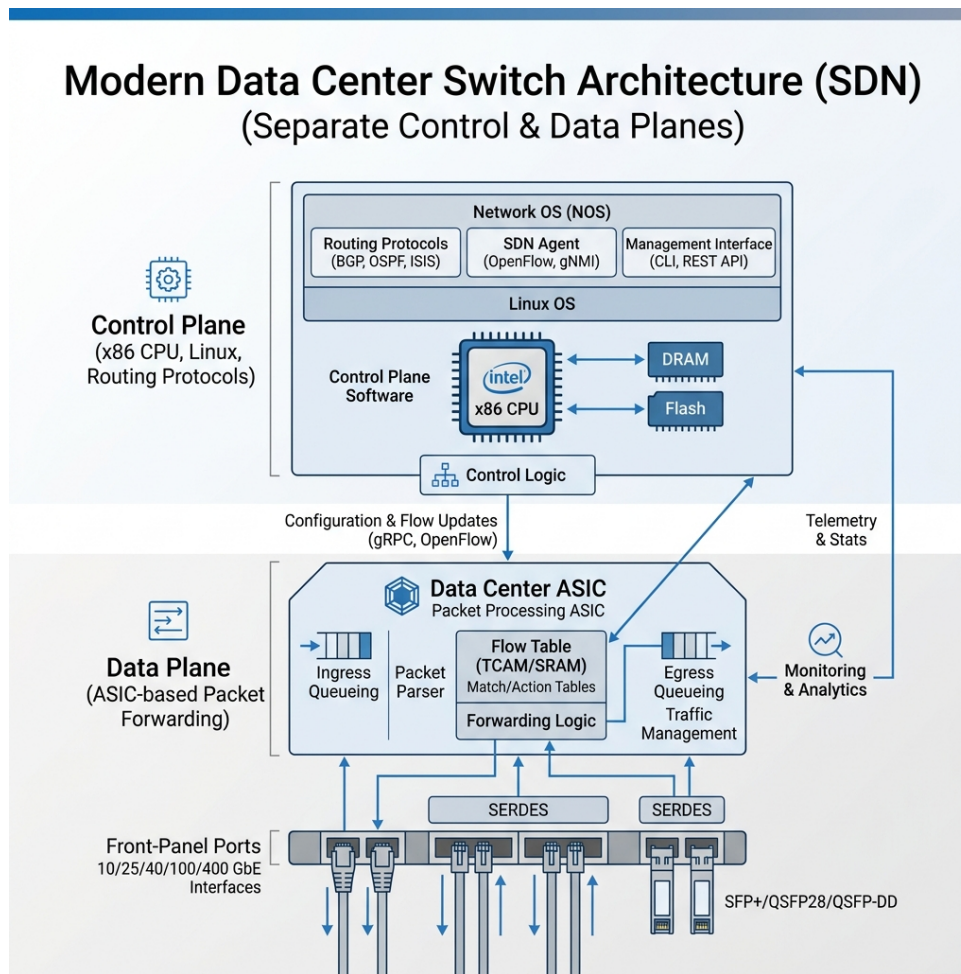


Figure 2.4: Architectural detail of the separation between ASIC (Data Plane) and x86 CPU (Control Plane).

1. **Data Plane:** dedicated hardware (often commodity ASICs, e.g., Broadcom chips) that *forwards* packets in a non-blocking way at *line-rate* speed. The introduction of a frame into this hardware ASIC adds a negligible latency of about **600 ns**.
2. **Control Plane:** an actual PC (often based on **Intel Celeron** or similar processors) integrated into the chassis, which manages network logic (BGP, OSPF). Being a full Linux PC, software can be compiled on it (e.g. running the VSCode editor directly inside the switch to edit scripts).

Running vs Startup Config

In network operating systems, managing the Control Plane memory is vital. Parameter modifications (e.g. via CLI) occur in the **Running Configuration** (located in volatile RAM); they take effect immediately but disappear in the event of a shutdown or blackout. To make them permanent, they must be saved to the **Startup Configuration** (flash memory/NVRAM) using the **write** (or **copy running-config startup-config**) command. Many catastrophic outages (impossible to debug later) arise from sudden reboots of devices whose configurations were never persisted to disk.

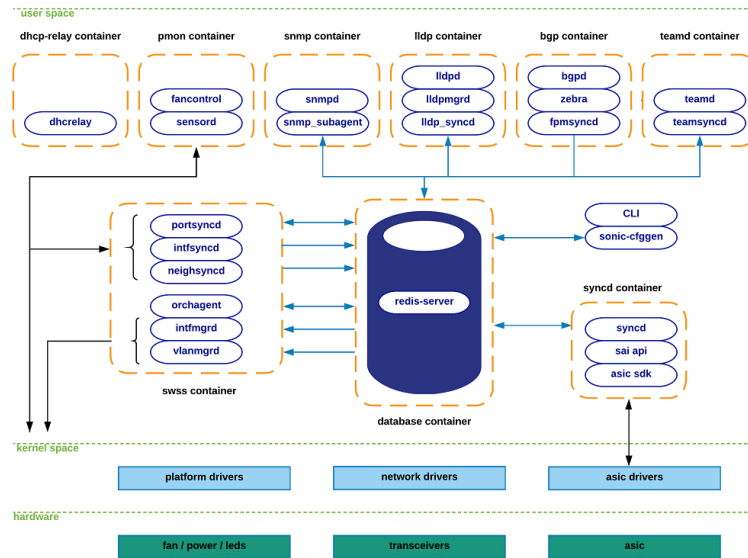


Figure 2.5: SONiC Architecture: each network function runs in its own Docker container communicating via Redis.

Hardware-software disaggregation led to the birth of open-source Network Operating Systems (NOS) like **SONiC** (Software for Open Networking in the Cloud, created by Microsoft). Booted via the ONIE bootloader, SONiC treats the switch like a standard server, running each routing daemon inside isolated *Docker containers*.

The OpenFlow API and the Firewall Sandwich

To instruct the Data Plane on how to forward packets, the Control Plane (or a centralized controller) uses communication protocols, the most famous of which is the **OpenFlow API**. This allows programmatic manipulation of the *Flow Tables* inside the ASIC. Thanks to this abstraction, architectures like the **Firewall Sandwich** are created: the SDN controller (OpenFlow) instructs the switch to divert all incoming traffic to a series of hardware firewalls placed in parallel (the "filling" of the sandwich). Once inspected, the traffic returns to the switch which routes it to its final destination, load-balancing and ensuring linear scalability of security, transparently.

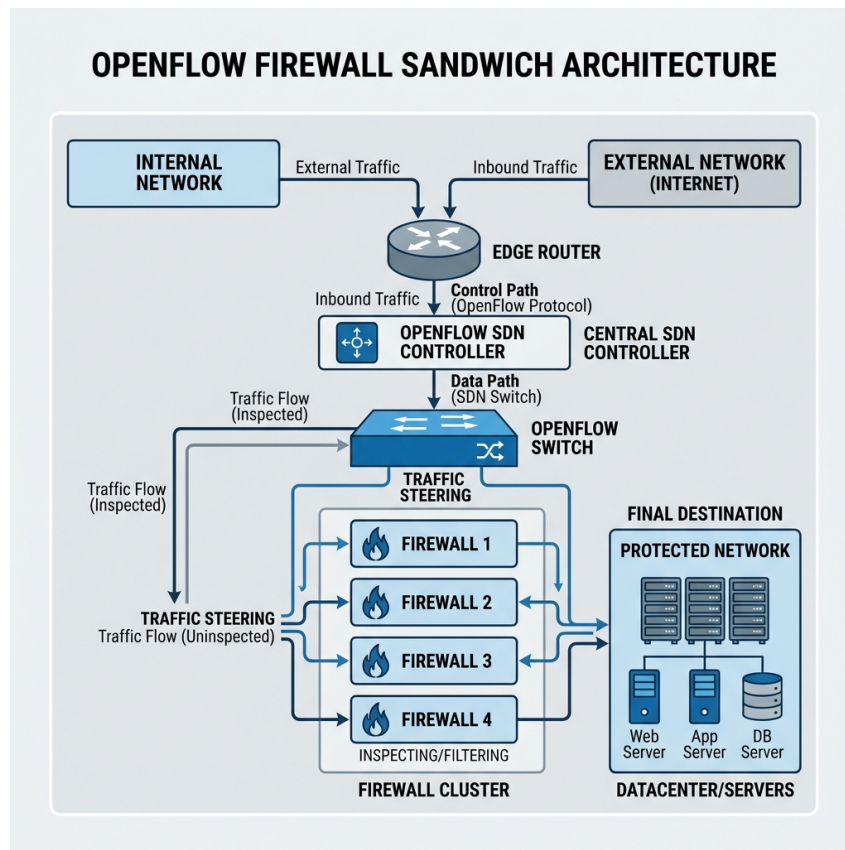


Figure 2.6: Firewall Sandwich architecture governed via SDN and OpenFlow.

2.3 Network Topologies: From 3-Tier to Spine-and-Leaf

Historically, datacenters adopted a **3-tier** topology (Access, Distribution, Core), optimized for **North-South** (client-server) traffic. Redundant links inevitably created physical loops. If a broadcast packet started to cycle, it triggered a catastrophic **Broadcast Storm** capable of paralyzing the entire datacenter in seconds. To prevent this, the **STP (Spanning Tree Protocol)** was used, which preemptively disabled redundant physical links. The flaws of STP were multiple: huge bandwidth waste (up to 50% of the laid cables remained dormant) and slow convergence times in case of failure (topology recalculations momentarily blocked traffic).

Today, the explosion of microservices, cloud, and AI has made **East-West** (server-to-server) traffic absolutely dominant. The standard topology of modern data centers has become **Spine-and-Leaf** (derived from *Clos* networks):

SPINE-LEAF NETWORK ARCHITECTURE

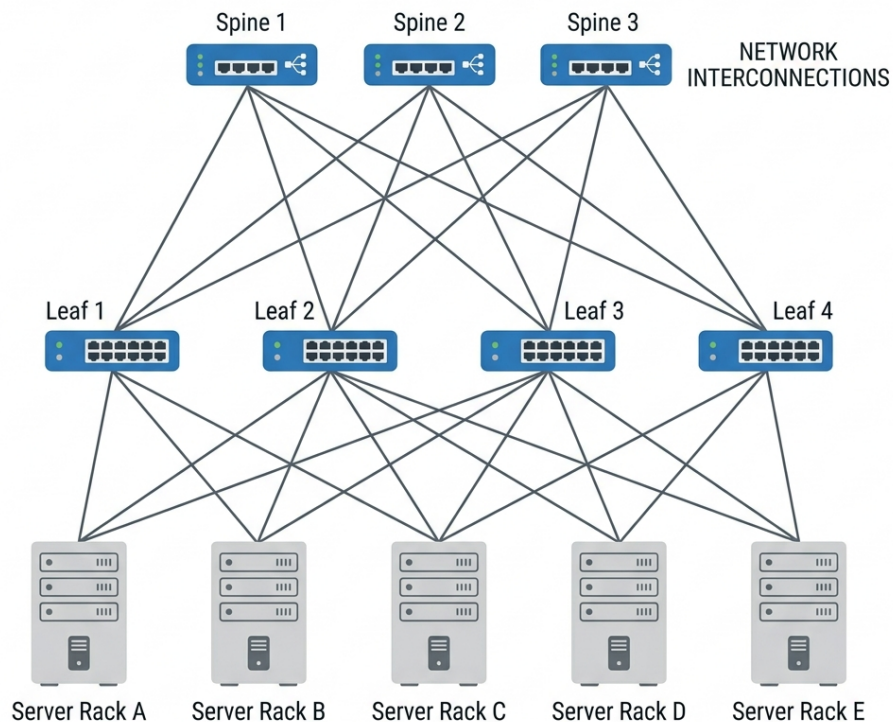


Figure 2.7: Spine-Leaf Architecture: Full-Mesh connection between Leaf layer and Spine layer.

The architecture is developed on two rigorous physical layers:

- **Leaf Layer (Access):** Consists of ToR (Top-of-Rack) switches. Physical servers connect *only* to these switches. In turn, Leaf switches **are never connected to each other**.
- **Spine Layer (Backbone):** The central aggregation layer. Every single Leaf switch has at least one fiber uplink to **every** single Spine switch (creating a *Full-Mesh* configuration between the two layers). Spine switches also **never talk directly to each other**.

Advantages and Oversubscription

- **Deterministic Latency:** Every server communicates with any other server taking exactly the same number of hops (1 logical hop: *leaf* → *spine* → *leaf*).
- **ECMP (Equal-Cost Multi-Path):** Without traditional physical loops, STP is replaced by L3 routing. Traffic is uniformly load-balanced across *all* available uplinks. If a Spine switch fails, the network degrades only fractionally in bandwidth, but does not go down.
- **Oversubscription Ratio:** The ratio between total downlink bandwidth (towards servers) and uplink bandwidth (towards Spines). A typical cloud uses 3:1 ratios. HPC or AI environments demand a **1:1** ratio (called *Non-blocking*), ensuring all servers can transmit at maximum concurrently without bottlenecks.

2.4 Network Segmentation and Virtualization

2.4.1 Jumbo Frames and MTU

Before logically fragmenting the network, it is essential to define its maximum payload. The standard **MTU (Maximum Transmission Unit)** in Ethernet is 1500 bytes. However, in datacenter environments (especially for storage traffic), segmenting huge files into 1.5 KB blocks causes unsustainable CPU overhead to process millions of headers. **Jumbo Frames** are therefore enabled, raising the MTU to **9000 bytes (9 KB)**. The operational trap is fragmentation: the MTU must be configured strictly identical on every single switch and server along the path, otherwise performance will drop drastically or packets will be silently lost (*black holing*).

2.4.2 VLAN (Virtual LAN)

The classic Ethernet header contains a destination MAC address (6 bytes), a source MAC (6 bytes) and an EtherType field (2 bytes). VLANs (IEEE 802.1Q standard) allow a physical network to be divided into multiple logical broadcast domains by inserting a 4-byte tag right after the source MAC. This 802.1Q tag includes a TPID field (indicating the presence of the VLAN) and a TCI, which in turn contains 3 bits for priority (QoS) and **12 bits for the VLAN identifier (VID)**. Since the VLAN ID is exactly 12 bits long, the unbreakable theoretical limit is **4094 usable networks** ($2^{12} - 2$), an absolutely insufficient number for modern multi-tenant cloud providers that must host hundreds of thousands of isolated networks.

2.4.3 VXLAN and Overlay

To overcome the architectural rigidity and the 4096-network limit of traditional VLANs, the modern cloud datacenter uses **VXLAN (Virtual eXtensible LAN)**. It is an *overlay* protocol (Mac-in-UDP) that encapsulates entire Layer 2 (Ethernet) packets within Layer 3 transport packets (UDP/IP). This raises the limit to **16 million isolated virtual networks** (thanks to a 24-bit VNI identifier) and allows the creation of arbitrary Layer

2 broadcast domains over a Layer 3 physical routing infrastructure. This means that two virtual machines can believe they are connected to the same physical switch (same L2 subnet), even if they are located in two physically distant L3-routed racks.

2.5 High-Performance Fabric (HPC and AI)

In HPC and AI training contexts, software overhead becomes the primary enemy. As seen, a frame crosses the switch hardware (the cut-through ASIC) in about **600 ns**. However, just to pass the data through the TCP/IP protocol software stack inside the server's operating system (interrupt handling, buffer allocation), about **70 μ s** of latency are introduced (over 100 times the switching time), a value that TCP inventor Vint Cerf associated with the problem of *bufferbloat* (the destructive accumulation of packets in intermediate buffers).

In these scenarios, **InfiniBand** dominates, a network that offers latencies of 1-3 μ s. The secret is the implementation of the **RDMA (Remote Direct Memory Access)** protocol, which allows a server to write directly into the RAM memory of another server, completely bypassing the intervention of the CPU and the receiving operating system. Today, RDMA is also available on advanced Ethernet networks via the **RoCE** (RDMA over Converged Ethernet) protocol.

Full Fat Tree

To avoid *overbooking* (uplink overload) critical for HPC, the **Full Fat Tree** topology is used: a non-blocking structure where the bandwidth towards the upper layers is exactly equal to that of the lower layers, ensuring bottleneck-free communications.

Chapter 3

Storage: Architectures and Services

3.1 Evolution of the Storage Medium

Storage represents the second fundamental pillar of a datacenter, the place where data persists across power loss (persistence). The absolute rule of storage is only one: **losing data is the unforgivable sin**.

3.1.1 From Magnetic Tape to HDD and NVMe SSD

Historically, the first datacenters used **Magnetic Tapes**: slow robotic cartridges but with enormous capacity and density. Tape forces a sequential access to data (you can't jump to the end without "unrolling" everything), but it is still used today for cold storage and offline backups impenetrable to ransomware (Air-gap).



Figure 3.1: Inside a traditional mechanical magnetic Hard Disk Drive.

Subsequently, there was a shift to **HDDs (Hard Disk Drives)**, which offer random access to data. However, they are limited by mechanical constraints (rotation of platters, head *seek* time), with impassable latencies of about 3-5 ms. Historical configurations for servers used complex cascaded interfaces (e.g., **SCSI**), today replaced by SAS or SATA protocols. Currently, HDDs are relegated almost exclusively to low-cost mass archiving.

The transition to **SSDs (Solid State Drives)** has reduced latency to a few hundred microseconds. However, the use of old SATA controllers quickly proved to be a bottleneck. The next revolution was the introduction of the **NVMe (Non-Volatile Memory Express)** standard, a protocol that connects flash memory directly to the **PCIe** bus, bringing latency down to $\sim 5\text{-}20\ \mu\text{s}$ and throughput to tens of GB/s.

Modern Memory Hierarchy

With NVMe, the performance gap between DRAM (nanoseconds) and permanent storage (microseconds) has shrunk from 6 to just 2 orders of magnitude, enabling software architectures unthinkable before, such as databases (e.g. RocksDB) that operate directly on storage without heavy caching layers in RAM.

3.1.2 NAND Cells and Asymmetry

Unlike DRAM, NAND Flash memory has an intrinsic asymmetry: read operations are fast, while writes require a much slower PE (Program/Erase) cycle. The bit density per cell determines the type of SSD:

- **SLC (Single-Level Cell):** 1 bit, extremely high endurance (100k PE cycles), extremely high cost.
- **MLC, TLC, QLC:** respectively 2, 3 and 4 bits per cell. Decreasing costs but rapidly declining endurance and write performance.

Data Retention and Over-provisioning

Two critical engineering problems afflict modern SSDs:

1. **Longevity (Data Retention):** To prevent the natural dissipation of electronic charge trapped in the cells, the SSD controller is forced to "silently" read and rewrite its NAND blocks in the background every ~ 2 weeks. Without prolonged power (e.g., in Cold Storage), data on SSDs evaporates.
2. **Over-provisioning:** An SSD cannot directly overwrite a block containing partial data; it must read, erase the entire block, and rewrite (Write Amplification). If an SSD exceeds 90% capacity, the free space for these maneuvers (*Garbage Collection*) runs out, causing a drastic and sudden drop in performance. To avoid this, pre-conditioning and hardware Over-provisioning are used (allocating a 10-20% extra flash space at the factory that is invisible to the operating system).

3.1.3 Performance Metrics: IOPS, Latency and Bandwidth

Latency and Bandwidth (throughput) alone are not sufficient to characterize the performance of a storage system, as they depend heavily on the access pattern (sequential or random). A fundamental metric is **IOPS** (Input/Output Operations Per Second), which is the number of I/O operations completed in a second.

- **Supermarket Analogy:** IOPS represents the customers served per second, latency is the time spent at the checkout for each customer, and parallelism (queues) are the open checkouts.
- The basic relationship for a single queue is: $\text{IOPS} \approx \frac{1}{\text{latency}}$.

- Thanks to the asynchronous nature of storage, protocols like NVMe (which support up to 64,000 parallel queues) allow to scale throughput immensely by processing many requests simultaneously.

3.2 Aggregation and Protection: RAID and Resilience

A single disk is an unacceptable single point of failure. **RAID** (Redundant Array of Independent Disks) is therefore used:

- **RAID 0 (Striping)**: No redundancy, extremely high performance, maximum risk.
- **RAID 1 (Mirroring)**: Identical copy (Mirrored LUN), loses 50% of the space but guarantees instantaneous availability in case of failure.
- **RAID 5**: Redundancy calculated via **XOR** logic function. Tolerance to the failure of only 1 disk. If a second disk fails during the delicate *rebuild* phase (which can take days on modern large disks), all data is lost.
- **RAID 6**: Uses double parity, allowing the simultaneous failure of **2 disks**.

The Twilight of Hardware RAID and Erasure Coding

The advent of NVMe SSDs (with throughputs of tens of GB/s) has paradoxically rendered expensive *Hardware RAID Controllers* obsolete. The controller chip cannot mathematically keep up with the parity calculation speed required for such fast disks, becoming the new bottleneck. Today, **Software RAID** dominates (e.g., `mdadm` on Linux or filesystems like ZFS), which delegates parity calculations to the immensely powerful multicore host CPUs.

In large-scale distributed cloud systems (like Object Storage), classic RAID fails. It is replaced by **Erasure Coding ($n=m+k$)**: a file is split into m data fragments, to which k mathematically calculated parity fragments are added. The fragments are scattered across tens of servers in different racks. It is possible to reconstruct the entire file having *any* m fragments available. This offers greater resilience than RAID at a lower storage cost, but with a higher CPU computational overhead.

3.3 Storage Architectures in the Datacenter

3.3.1 SAN (Storage Area Network)

SAN centralizes storage by separating it from servers, communicating at the block level (exposes raw disks, not files). The dominant protocols for this fabric are:

- **Fibre Channel (FC)**: Predominant legacy standard before the NVMe era, it relies on the **SCSI** protocol for block movement. It is a physically and logically separate network from standard Ethernet, often built with a physically redundant dual fabric topology (Fabric A and Fabric B). It requires dedicated cards on servers, called **HBA (Host Bus Adapters)**. It identifies nodes not by MAC addresses, but by 64-bit addresses called **WWN (World Wide Name)**.

- **iSCSI:** Economic alternative that transports SCSI commands inside normal TCP/IP packets over an Ethernet network. It works via a client-server architecture: the host server acts as an **iSCSI Initiator** (which requests blocks), while the storage array is the **iSCSI Target** (which provides them). Nodes are identified via **IQN** strings.
- **FCoE (Fibre Channel over Ethernet):** The historical attempt to unify the two networks. It encapsulates FC frames (SCSI) directly inside standard Ethernet (Lossless Ethernet using DCB/PFC protocols), eliminating the need for separate cabling, without the overhead of TCP/IP. Today it is almost entirely superseded by **NVMe-over-Fabrics (NVMe-oF)**.

SAN Security: Zoning and Masking

To prevent one server from accidentally formatting (or intercepting the data of) another server's disk, two barriers are used:

1. **Zoning (Network Side):** Applied on switches, it creates closed hardware zones. Similar to the concept of VLANs in Ethernet, it prevents two HBAs that are not in the same zone from communicating or "seeing" each other.
2. **LUN Masking (Storage Side):** The storage array authorizes access to a logical disk (**LUN, Logical Unit Number**) exclusively to the specific WWN (for FC) or IQN (for iSCSI) address of the owner server.

3.3.2 NAS (Network Attached Storage)

NAS operates at a higher abstraction level, connected via standard Ethernet network (NFS or SMB protocols).

- **Abstraction:** Exposes a logical File System (*files and directories*).
- **Limits and mmap:** Higher network latency. In particular, NAS destroys application performance in case of massive use of the **memory mapping (mmap)** directive for random access (e.g. databases or VM virtual disk files). *mmap* maps a logical disk file directly into the RAM space of a process (Page Cache). If the file resides remotely on a NAS, every single *page fault* (reading a missing fragment in RAM) synchronously triggers an entire blocking Ethernet network round-trip, masking a disastrous I/O delay behind a simple memory instruction, leading to application stalls (*stalls*). This problem does not occur with SAN block storage (direct DMA).
- **Use Cases:** File sharing, backups, corporate documents.

3.3.3 Object Storage

Developed to handle exabyte-scale scalability (e.g. Amazon S3). It abandons folders and blocks, using flat **objects** allocated in *buckets* and queryable via REST APIs. To ensure scalability and distribute data without a central bottleneck, it uses routing algorithms like **Consistent Hashing**. Ideal for massive static data, such as Machine Learning datasets or medical archives.

3.3.4 HCI (Hyper-Converged Infrastructure)

For decades the datacenter market was dominated by large vendors providing proprietary hardware SAN and NAS (e.g., the famous **EMC Symmetrix** arrays or **NetApp Filers**), extremely expensive appliances with difficult horizontal scaling. To cut costs and scale linearly, there was a shift to **HCI**, which transforms centralized SAN storage into a distributed approach (*Scale-out*). Every server (commodity x86 node) contains CPU, RAM and local NVMe disks. A software layer (Software-Defined Storage) pools all local disks of all nodes and presents them to the hypervisor as a single large shared virtual SAN.

HCI Advantage

Data Locality: The software always attempts to write/read the data of a VM in the disk physically residing in the same server that is running it, zeroing out North-South network traffic and fully exploiting local PCIe bandwidth.

3.3.5 Storage for AI: Vector Database

The emergence of generative Artificial Intelligence required new storage paradigms. Classic textual search was purely *syntactic* (like the *bag of words* model), while neural models operate on *semantics*.

- **Embeddings:** Neural networks (LLMs) transform text, images or audio into very high-dimensional dense vectors (e.g. 3000 dimensions). These vectors, called *embeddings*, represent the actual meaning of the data based on context (e.g., distinguishing "Python" as a language from "Python" as a snake).
- **Vector Database:** These are a new generation of NoSQL databases specifically designed to store and index these huge vectors. To answer a query, they vectorize the user's question and look for the most semantically similar documents by calculating the **cosine similarity** in the vector space.

RAG: Retrieval-Augmented Generation

Vector Databases are the engine of the **RAG** architecture, an essential technique to limit AI "hallucinations". Before answering a prompt, the system queries the vector database, retrieves truly relevant corporate documents, injects them into the prompt, and forces the LLM to formulate the response based solely on those verified facts.

3.4 Advanced Services and Resilience

3.4.1 Efficiency: Deduplication, Compression and Tiering

In addition to storing blocks, modern storage controllers (NAS/SAN) offer value-added services (often *in-line*, computed in real-time during writing via dedicated ASIC chips):

- **Deduplication:** Identifies and removes physically identical data blocks (e.g., if there are 100 Windows virtual machines with the same OS file, the data is saved only once on disk). A logical reference counter tracks the copies for the hosts.

- **Compression:** Mathematical reduction of the data footprint on disk.
- **Caching and Auto-Tiering:** Automatic promotion of the most read/written data blocks ("hot data") from slow disks to very fast tiers (e.g. NVRAM or NVMe SSD) completely transparent to the application.
- **Multipath I/O:** At the server level, each HBA is connected to parallel network fabrics (e.g., switch A and B) to have simultaneous logical paths to the same LUN of the storage array, ensuring load-balancing and instantaneous failover (MPIO).

3.4.2 Snapshot, Thin Provisioning and Replication

- **Thin Provisioning:** Allows logically allocating more space to users than is physically present in the array, based on the principle that allocated space is rarely filled (*Overbooking*). The fatal risk is physically exhausting free space if everyone writes, causing a systemic panic block of the entire datacenter (no one can write to disk anymore).
- **Snapshot vs Clone:** A *Snapshot* is an instantaneous photograph of the volume at zero cost in terms of space. It exploits the **Copy-on-Write (COW)** mechanism: it freezes the original blocks and allocates new blocks (differential files) only for subsequent modifications. **Impact on performance:** a high chain of active snapshots heavily degrades read I/O performance. A **Clone**, instead, is a complete bit-by-bit physical copy (consumes space and time).
- **Crash-consistent vs Application-consistent:** A basic snapshot is always *crash-consistent* (equivalent to brutally pulling the plug, without flushing RAM caches), which inevitably corrupts databases. To prevent this, the infrastructure uses agents like **Windows VSS (Volume Shadow Copy Service)** that coordinate with the OS to briefly "freeze" application I/O (*quiesce*), take the snapshot and resume in an *application-consistent* manner.
- **Synchronous/Asynchronous Replication:** Essential for Disaster Recovery. In the case of geographically close sites (within 15ms latency), **Metro Clusters** with synchronous active-active replication are used.

3.4.3 Backup vs Replication

Replication is a real-time mirror copy: it protects against hardware failures, but instantly propagates human deletions or ransomware attacks. **Backup** is a disconnected (offline) point-in-time save that allows recovery.

In the sizing of a business continuity system, two metrics are fundamental:

- **RPO (Recovery Point Objective):** Amount of data the company is willing to lose (backup frequency).
- **RTO (Recovery Time Objective):** Maximum tolerable time for infrastructure recovery.

GDPR and Data Breach

According to GDPR, a *data breach* is not only the theft of data (loss of *confidentiality*), but also includes the loss of *integrity* (corrupted data) and **availability** (data intact but inaccessible, e.g. during a ransomware attack encrypting the only primary storage).

Chapter 4

Compute and Virtualization

4.1 Compute Architecture: CPU, GPU and NPU

Data processing (Compute) is the third pillar of the datacenter. Modern hardware evolution is driven by the end of linear single-core scalability (Moore's Law) and the need for efficiency and specialization for Artificial Intelligence workloads.

4.1.1 Energy Sustainability

Datacenters consume huge amounts of electricity and water. The benchmark metrics are:

- **PUE (Power Usage Effectiveness):** Ratio between the total energy consumed by the datacenter and that actually used by the servers. The ideal value is 1.0.
- **WUE (Water Usage Effectiveness):** Water consumption per kWh, critical in datacenters using evaporative cooling (Cooling towers).

4.1.2 CPU Architecture: Pipeline and Multi-Core

Historically, to increase performance, clock frequency was increased and the **Instruction Pipeline** was optimized (classically divided into phases: *Instruction Fetch, Decode, Execute, Memory Access, Write-back*). By increasing pipeline depth (up to 20+ stages), more overlapping instructions could be executed over time, but *Branch Misprediction* errors caused stalls and complete queue flushing ("bubbles" in the pipeline), nullifying efficiency.

Having reached the physical limit of dissipatable heat (Power Wall), the shift was made to Multi-Core parallelism. To overcome RAM sharing bottlenecks, modern CPUs adopt **NUMA (Non-Uniform Memory Access)** architectures, where each processor has its own local memory bank with fast direct access; access to the RAM of other processors occurs via external interconnects with higher latency.

Hyperthreading and SMT (Simultaneous Multithreading)

To minimize idle times when a core waits for data from memory, designers introduced **Hyperthreading** (or SMT). The trick is to duplicate only "lightweight" resources (like registers and the program counter), while sharing the expensive physical calculation units (ALU). The operating system sees **two logical cores** for each physical core: when a thread stalls for a *cache miss*, the core instantly switches context and executes the other thread's instructions. Although excellent for web servers and high I/O databases, HT is

harmful in HPC (High Performance Computing): if two threads must perform intensive simultaneous mathematical calculations, they will fiercely fight for the single shared ALU, reducing overall performance. Therefore, in supercomputers, HT is disabled.

Overbooking and Memory Ballooning

Virtualization relies on **Overbooking** (logical over-allocation of underlying physical resources). In addition to the obvious over-commit of vCPUs, heavy RAM overbooking is performed. A key mechanism is **Memory Ballooning**: the hypervisor injects a balloon driver into the guest OS kernel. If the physical server is running out of real RAM, the hypervisor "inflates" the balloon. The guest OS believes it urgently needs internal memory, swaps its least used processes to its virtual disk, and returns the real RAM to the balloon (and therefore to the hypervisor). The hypervisor also masks huge amounts of RAM via *Memory Page Sharing* (e.g., KSM deduplicates identical operating system pages across dozens of identical Windows VMs).

Side-Channel Vulnerabilities

Physically sharing the L1 cache and execution buffers between two threads opens the door to speculative attacks like **Spectre** or *MDS*. A malicious thread can infer the "sibling" thread's secret data by measuring cache latency. For maximum security, some cloud providers disable SMT entirely.

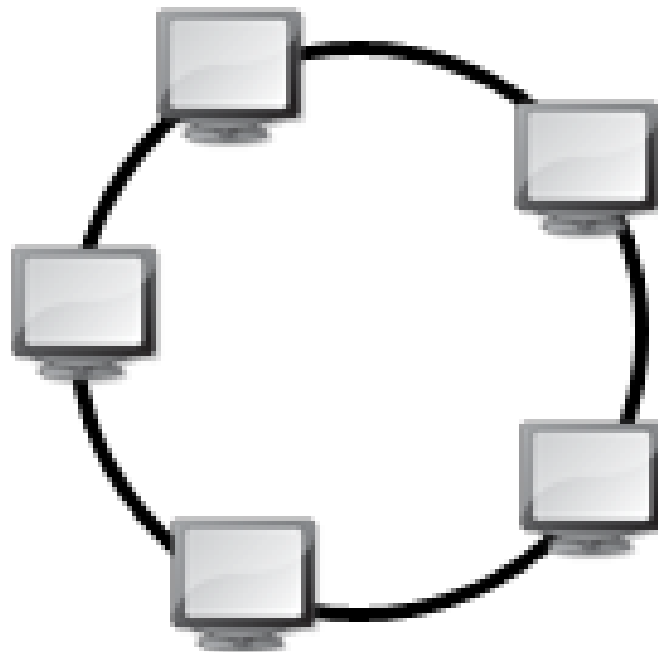


Figure 4.1: Logical diagram of a Ring topology, historically used for internal CPU core interconnection.

Internal Interconnect: Crossbar vs Bus

At the internal interconnection level (die), early multi-core CPUs used a *Shared Bus*, which immediately became a bottleneck: only one core at a time could talk. Today, the

Crossbar (or a *Mesh Network*) is used: an interconnection matrix that allows multiple cores to communicate simultaneously with different memory banks or I/O without interfering, ensuring full *bisection bandwidth*.

This reticular approach pairs perfectly with the **Tile (or Chiplet)** architecture: modern processors are no longer a single silicon monolith prone to defects, but a mosaic of small hyperspecialized chips (Compute Tile, I/O Tile) printed with different nodes and glued onto the same package, vastly improving production yields and lowering costs. Concrete examples of this approach are the **AMD Infinity Fabric** interconnect fabric used to join chiplets in EPYC processors, and advanced 3D packaging technologies like **Intel EMIB and Foveros** (used in Meteor Lake and Sapphire Rapids architectures) that stack silicon dies vertically.

4.1.3 GPU and NPU: The Paradigm Shift

- **CPU:** Optimized for *minimum latency*. Few very complex cores with *Out-of-Order* execution and large caches.
- **GPU:** Optimized for *maximum throughput*. Thousands of simple cores executing the same calculation in parallel (SIMT), ideal for AI training.
- **NPU (Neural Processing Unit):** Dedicated circuits specialized exclusively in tensor calculation and AI inference at low numerical precision and ultra-low power consumption.

4.2 Physical Server Architectures and BMC

The physical enclosure (form factor) of servers has diversified over the years to meet different needs for density, computing power, and thermal dissipation.

4.2.1 Rack Servers (1U / 2U)



Figure 4.2: Standard 1U Rack Server with front hot-swap disk bays.

It is the most common "general-purpose" format in traditional data centers. A rack server is a complete and independent computer (equipped with its own power supplies, fans, CPU, RAM and disks) enclosed in a standardized 19-inch wide metal chassis. The height is measured in rack units (U, about 4.4 cm). They are ideal for heterogeneous workloads, but in large quantities they require an enormous amount of individual cabling work for each machine (separate power and network cables for each individual server).

4.2.2 Blade Servers



Figure 4.3: Blade System Chassis: dozens of server blades inserted into a single module.

Designed to achieve the ultra-high density needed in virtualization clusters. A Blade system is divided into two components: the **Chassis** (the outer enclosure providing shared power, mega-fans and integrated network switches) and the **Blades** (the actual servers). The Blade is very thin because it only contains CPU, RAM and a few disks; it has no fans or power supplies of its own. By inserting it into the chassis, it automatically connects to the rear *midplane*, almost completely eliminating cabling clutter in the rack.

4.2.3 HGX Servers (AI GPU Server)



Figure 4.4: HGX Server (open): high-performance GPUs covered by massive metal heatsinks to handle extreme heat.

With the explosion of generative artificial intelligence, regular rack servers are no longer sufficient. An HGX server is a massive machine (often 4U or 8U, weighing over 100 kg) designed exclusively around GPUs (e.g., NVIDIA H100). GPUs are no longer inserted as removable PCIe cards, but are soldered directly onto a proprietary *baseboard* and interconnected via dedicated buses (like NVLink) that bypass the PCIe bottleneck. They generate frightening heat, often making the switch to liquid cooling (*Direct-to-Chip Liquid Cooling*) mandatory.

4.2.4 Hyperscaler Standardization: Open Compute Project (OCP)

Originally created by Meta (Facebook) in 2011, the **OCP (Open Compute Project)** is a non-profit consortium that publishes open source hardware specifications for datacenter servers and racks. The idea stems from an economic need: traditional branded servers contain many aesthetic or management features (plastic bezels, displays, redundant power supplies per single server) that are useless on a hyperscaler scale. OCP specifications define essential machines (often produced directly by Taiwanese ODM manufacturers like Quanta or Foxconn), allowing for enormous savings. A key OCP innovation is the rack design: power is not converted in every single server, but centralized upstream in the rack and distributed to the servers via 12V or 48V busbars, increasing energy efficiency and density.

BMC (Baseboard Management Controller) and the Redfish Standard

All enterprise-grade servers are equipped with a **BMC** (e.g. HPE's iLO or Dell's iDRAC). It is an independent micro-computer inserted into the motherboard, equipped with a dedicated network port (*out-of-band management*). Being separately powered, it allows physical control of the server (power on, BIOS configuration, virtual KVM console) even if the main operating system is frozen or if the server is physically powered off.

Redfish: The Modern Datacenter API

Historically, BMCs were managed via the IPMI protocol, which over time proved to be insecure and difficult to integrate. For this reason, the DMTF consortium introduced **Redfish**, the modern standard for infrastructure management.

- **How it works:** Redfish exposes all hardware parameters via a standard **RESTful API**. It uses modern web protocols (HTTPS for security) and returns data in **JSON** (OData) format, easily readable by both humans and scripts.
- **What it does:** It allows automating the most tedious operations on a large scale. Through simple Python scripts or tools like Ansible, web requests (GET, POST, PATCH) can be sent to monitor temperatures, update firmware, change the boot order or restart an entire rack of servers, regardless of the underlying hardware brand.

4.3 Virtualization and Live Migration

Virtualization decouples software from physical hardware, allowing multiple Operating Systems (Guests) to run simultaneously on the same physical server (Host). The heart of this process is the **Hypervisor** (or Virtual Machine Monitor).

4.3.1 Hypervisor Evolution: From CPU Rings to Hardware Virtualization

The original x86 architecture was not designed to be virtualized. It provided four logical privilege levels called **Rings**, from Ring 0 (maximum privilege, intended for the OS kernel) to Ring 3 (user space). In a virtualized environment, the Hypervisor must have absolute control, therefore it must reside in Ring 0. This forced Guest operating systems to scale into Ring 1 (unprivileged mode, defined as *Ring De-privileging*). Since Guest OSes did not know they were virtualized, they attempted to execute privileged instructions (e.g., hardware manipulation) which, failing, were trapped by the Hypervisor. The latter had to translate and emulate them in real time (**Binary Translation** technique), causing huge and unsustainable performance overhead.

The problem was solved in 2005 by CPU manufacturers with the introduction of dedicated hardware extensions (**Intel VT-x** and **AMD-V**). These instructions created a new privilege level below zero (often colloquially called **Ring -1**). Today the Hypervisor resides in Ring -1, allowing Guest operating system kernels to operate in their native Ring 0, executing instructions directly and at hardware speed, eliminating the need for slow binary translation.

4.3.2 Hardware Abstraction: vCPU, Pause and Resume

The virtual machine does not see the real physical CPU, but a **vCPU** (Virtual CPU) configured by the hypervisor. This is a crucial detail for portability: the hypervisor can hide overly recent instruction sets (e.g. AVX-512) by masking the true capabilities of the physical processor, so that the VM can be freely migrated even to an older server that does not possess those instructions.

Furthermore, since the entire Guest environment is just software running on a controlled *fetch-execute* cycle, the hypervisor enjoys the power of **Pause and Resume**. It can instantly freeze the operating system by saving *the complete state of the VM* (CPU registers, memory state and virtual disk). The virtualized OS will notice nothing; for it, time will have simply stopped. Through **Integration Services**, the hypervisor then forces clock synchronization upon resumption (vital for protocols like Kerberos which fail if the time is desynchronized), sends *heartbeat* signals to understand if the Guest kernel has crashed, and orchestrates graceful shutdowns.

Windows Dual Boot and Secure World with WSL2

Today virtualization is pervasive even on client devices. Upon startup, a modern Windows PC (VBS enabled) does not perform a normal bare-metal boot, but launches a hypervisor (Hyper-V) that instantiates two environments. The first is a hidden micro-OS (**Secure World**) that isolates cryptographic hardware (e.g. BitLocker keys). Immediately after, the main desktop Windows environment is launched, which therefore runs *virtualized*. Even if a hacker obtains maximum privileges (Ring 0) on the Windows kernel, they can never steal the Secure World's cryptographic keys. The same transparent client hypervisor makes **WSL2** (Windows Subsystem for Linux) possible, which runs a true, complete Linux kernel in an ultra-light micro-VM alongside Windows.

4.3.3 The Virtual Switch (vSwitch)

To connect VMs to the outside world, the Hypervisor implements a **Virtual Switch** in RAM. Virtual machines possess logical network interface cards (**vNICs**), which connect to the vSwitch ports. In turn, the vSwitch routes traffic to the physical network via the server's real network interface cards (**pNICs**). The vSwitch manages complex logic in software such as VLAN tagging, traffic shaping, and physical interface teaming in a way that is completely transparent to the guest virtual machines.

4.3.4 Live Migration

The most disruptive feature of virtualization is **Live Migration** (e.g. VMware vMotion), which allows moving a running VM from one physical host to another without service interruption. It rigorously develops in three sequential phases:

1. **Pre-copy (Dirty tracking)**: Initial massive memory transfer while the VM runs on the old host. The system actively tracks via a bitmap (*dirty page tracking*) which memory pages are modified during the slow network copy operation. This cycle is repeated to transfer subsequent deltas.

2. **Stop and copy:** When very few dirty pages remain, the VM is "frozen" (imperceptible pause of a few ms). The residual state of CPU registers, I/O buffers, and residual RAM is copied. Execution is transferred to the destination hypervisor.
3. **Network Management (Gratuitous ARP):** Since the VM's MAC address has physically moved to another port of the datacenter core switch, the destination vSwitch sends a broadcast **Gratuitous ARP**. This instantly updates the CAM tables of all physical switches, ensuring that in-transit packets are not lost.

4.4 Containerization

Unlike virtualization, **Containers** do not emulate entire hardware or an entire kernel, but **share the host's Linux kernel**. This reduces overhead to zero, but radically changes security and networking modes.

4.4.1 Isolation Mechanisms and Vulnerabilities

Isolation, vital to prevent containers from collapsing into each other, is delegated to the operating system kernel via two primitives:

- **Namespaces:** Isolate logical visibility. A namespace makes the container believe it is the only system, giving it an independent PID tree and exclusive mount points.
- **Cgroups (Control Groups):** Limit hardware usage (impose strict CPU ceilings or strict *OOM killers* for RAM).

Sharing the kernel means, however, that a flaw in it affects everyone simultaneously. The course cites the anomalous case of the **CVE-2026-31431 ("Copy Fail")** vulnerability in the `authenc:esn` module: by abusing the *page cache* shared between containers, a trivial Python script allows not only privilege escalation, but a catastrophic *container escape*, jeopardizing entire orchestration nodes.

4.4.2 Docker Architectural Details

- **Overlay File System:** Containers use layered file systems (e.g. *overlay fs*). The container image is immutable (read-only). By starting it, a lightweight ephemeral *Read-Write* layer is created on top. Modifying a base file triggers a slow *Copy-on-Write* to the upper layer.
- **Multi-stage Build:** In modern software development, a giant container is used to compile source code. Then, a second stage copies only the final compiled binary into an empty image (e.g. 5MB Alpine Linux), drastically reducing the footprint and attack surface.
- **Hardware Drivers distributed via Container:** Even complex drivers are being "containerized". NVIDIA, to facilitate AI deployments on bare-metal nodes, distributes the NVIDIA Container Toolkit, encapsulating GPU drivers and CUDA libraries in an isolated container instead of installing them on the host.

4.4.3 Network: VM vs Container

In virtual machines, the hypervisor creates an isolated **vNIC** (a real fake card visible to the VM) and connects it to the vSwitch. In containers, this abstraction is missing. Instantiated containers (if they don't use complex bridges like in K8s) **directly contend for access to the pNIC** (physical card of the host) via internal routing. For this reason, containers need **Port Mapping**: the internal port of the daemon (e.g., NGINX's 80) must be mapped and translated via NAT to another exposed port on the host server.

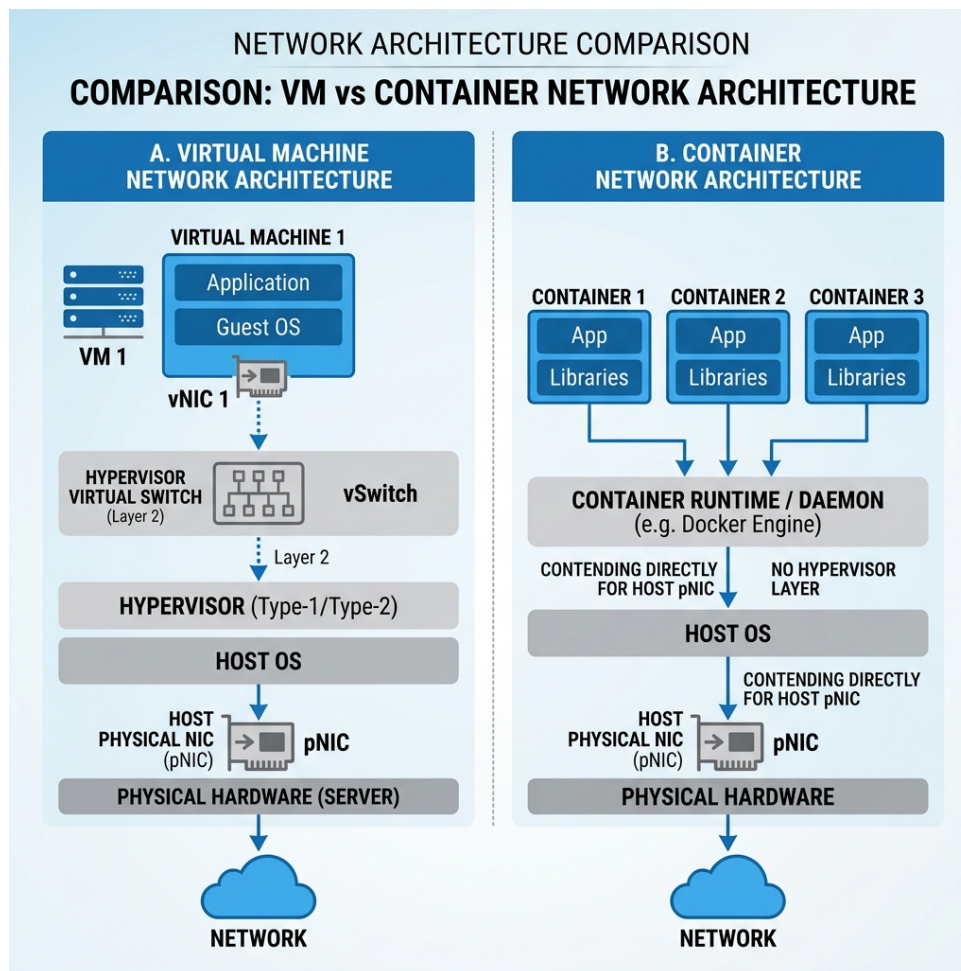


Figure 4.5: Network architecture: vNIC abstraction (VM) vs contention and NAT (Container).

The integration of Containers and Virtualization is, however, the standard: the cloud often runs Kubernetes clusters composed of *Worker nodes* that are themselves virtual machines, thus combining the lightness and deployment ease of containers with the robust hardware isolation (preventing container escape) of VMs.

Chapter 5

Cloud Computing, Orchestration and Security

5.1 The Cloud Computing Model

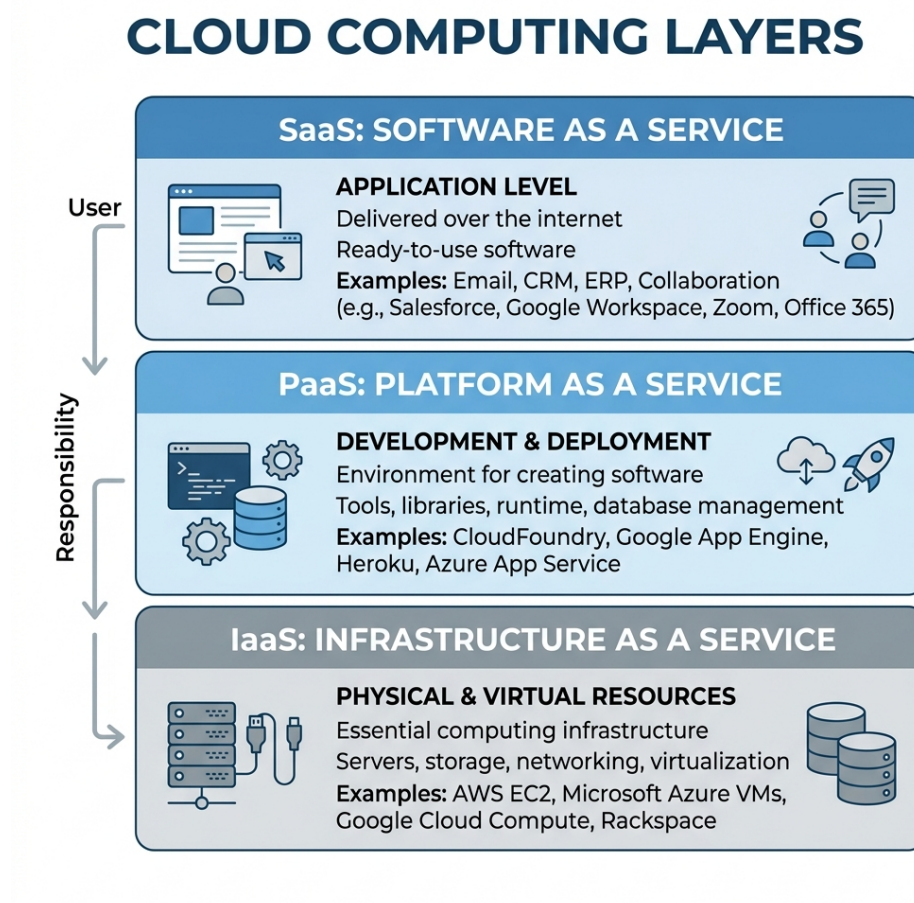


Figure 5.1: Abstract representation of Cloud Computing architecture layers.

Cloud computing represents not only a technological change, but a profound shift in the IT business model: moving from purchasing hardware (CapEx) to paying for the consumption of a service (OpEx).

The Challenges of Public Administration

Building a cloud for a digital-native company or for the Public Administration (PA) are two opposing challenges. A startup operates in a **Greenfield** environment (starting from scratch with modern standards). The PA typically operates in a **Brownfield**: it must migrate decades-old applications and mainframes to the cloud without interrupting critical services to citizens. Furthermore, structural obstacles intervene:

- **Governance:** Centralized model (more secure and economical on a national scale) vs Federated (politically preferred by local entities that do not want to cede control over their data).
- **Chargeback:** How to exactly allocate costs (internal billing) among different departments/ministries sharing the same hardware without generating disputes.
- **Procurement:** Purchasing physical hardware in PA on-premise clouds requires exhausting public tenders (with average delays of 1-2 years), making the purchased server already technologically obsolete by the time it is physically delivered to the datacenter.

5.1.1 Definition and Characteristics (NIST)

According to NIST, the cloud is characterized by:

- **On-demand self-service:** Autonomous provisioning by the customer without human intervention from the provider.
- **Broad network access:** Ubiquitous accessibility via standard network.
- **Resource pooling:** Shared resources in multi-tenant mode.
- **Rapid elasticity:** Ability to rapidly scale up and down (resources appear infinite to the consumer).
- **Measured service:** Pay-as-you-go model and continuous measurement of resources.

5.1.2 Service Models

- **IaaS (Infrastructure as a Service):** The provider supplies "raw" or virtualized CPU, network, and storage. The consumer manages from the Operating System upwards.
- **PaaS (Platform as a Service):** The provider also manages the OS and runtime environment. The consumer only loads their application (e.g., managed databases, web apps).
- **SaaS (Software as a Service):** Everything is managed by the provider (e.g., Gmail, Microsoft 365). The user only uses the final software.

5.2 Reference Architecture (Reference Model)

The Reference Model is articulated in layers:

1. **Physical Layer:** Physical hardware (Servers, Switches, Storage arrays). At this level operate the **Element Managers**, which are proprietary management interfaces provided by the manufacturer (e.g. the GUI of a single switch or an EMC array) that see *only* their own physical device.
2. **Virtual Layer:** Virtual machines, vSwitches, virtual LUNs. Abstracts the physical hardware providing flexible resource pools.
3. **Control Layer:** Responsible for globally managing and grouping virtual resources. The **Unified Manager** operates here (e.g., VMware vCenter or Microsoft SCVMM), a super-manager that aggregates thousands of hosts ignoring hardware brand differences and allows the administrator to apply allocation *policies* (absolute vs relative reservations) and over-allocation (Overbooking).
4. **Orchestration Layer:** Automates complex workflows via scripts and asynchronous API calls. While the control layer executes a single primary task ("turn on VM"), the orchestrator coordinates an entire multi-step *pipeline* ("create VM, configure vSwitch, allocate IP, install OS, install Database").
5. **Service Layer:** Provides the *Cloud Portal* and *Service Catalog* in self-service mode, from which end users (tenants) order resources with a click, without having to open IT tickets.

5.2.1 Cloud Service Lifecycle

Behind the scenes, the delivery of a cloud service passes through four mandatory macro-phases:

- **Phase 1: Service Planning.** The IT architect defines the boundaries of the service, calculates the required resources and, above all, defines the **Billing Policy** (the exact billing metric, e.g., per allocated GB, per CPU-hour or per Giga of outgoing traffic).
- **Phase 2: Service Creation.** The orchestrator receives the order from the portal and converts it into dozens of concrete API calls to the infrastructure to build the service, allocating IP, disks and CPU.
- **Phase 3: Service Operation.** The service goes into production and is handed over to the tenant. It is constantly monitored by probes to verify compliance with SLAs (Quality of Service) and to quantify the consumed resources for billing purposes (Metering).
- **Phase 4: Service Termination.** At contract expiration or cancellation, the system must automatically deallocate all resources, irreversibly delete data (Secure Wipe) to protect the old tenant's privacy, and return IP addresses and storage to the available pool.

SLA (Service Level Agreement) and Legal Clauses

It is the binding contract between cloud provider and consumer that establishes the guaranteed service levels (uptime, performance, security). It includes measurement metrics and penalties (*Service Credits*) in case of violation. Historically, SLAs hide complex clauses in favor of the provider:

- **Downtime vs User-minutes:** The calculation of downtime is not absolute. A failure of 10% of the infrastructure might not violate the SLA if calculated on the total expected monthly "user-minutes".
- **Exclusions:** Scheduled maintenance (*Scheduled Downtime*) or outages caused by customer misuse are excluded from penalties.
- **Lock-in:** Technical-legal difficulties in migrating one's data out without incurring enormous transfer costs (*Egress*) or dependency on proprietary formats.

5.3 Business Continuity and Data Protection

The cloud assumes that hardware failures are physiological. The goal is to minimize the impact on the service (High Availability). The main parameters are:

- **RTO (Recovery Time Objective):** Time needed to restore the service in case of disaster.
- **RPO (Recovery Point Objective):** How far back in time data is lost in case of restoration. Determines backup frequency.

5.3.1 Resilience Techniques and Geographical Limits

To avoid **SPOFs (Single Points of Failure)**, multi-level redundancy techniques are used:

- **Service Failover vs VM Replication:** Traditional *Service Failover* (e.g. Windows Server Failover Cluster) relies on a passive secondary node; if the primary dies, the secondary mounts the shared storage and performs a slow cold reboot of the application (with an RTO of minutes). *VM Replication* or **Continuous Data Protection (CDP)** operate instead at a lower level. A CDP injects a driver (*Write Splitter*) into the hypervisor that bifurcates every I/O write: one copy goes to the local disk, the other via asynchronous network to the Disaster Recovery site. It uses a *Journal Volume* to maintain a granular second-by-second timeline, allowing to "rewind" the entire VM in time to 5 seconds before a potential ransomware attack.
- **Active/Active vs Active/Passive:** The Active/Passive approach uses a standby secondary node. The Active/Active approach distributes the load simultaneously across multiple geographic datacenters, but encounters a severe physical limit dictated by the speed of light in optical fiber.
- **Synchronous vs Asynchronous Replication:** For two datacenters to operate in true Active/Active (without corrupting shared databases), storage replication

must be **Synchronous** (the write is confirmed to the client only after both geographic sites have saved it). To do this without paralyzing applications, the latency time (*Round-Trip Time*) between datacenters must not exceed **10-15 milliseconds**, which forces sites to be no more than $\sim 100\text{-}150$ km apart from each other. For greater distances (e.g., between continents), it is mandatory to use **Asynchronous** replication (where there is a vulnerability window, $RPO > 0$), because waiting for the light signal would render the system unusable.

- **Availability Zones:** Division into distinct zones (often in different datacenters within the 15ms threshold) isolated from common failures (power, floods) but connected by dedicated optical fiber (Dark fiber) to appear as a single large logical infrastructure.

5.3.2 Application Resiliency: Design for Failure

Modern cloud architectures surrender to the statistical evidence that hardware will inevitably fail. Resilience is therefore abstracted and moved entirely to the software level (**Design for Failure**):

- **Statelessness and Persistent State:** Containers and web servers must *never* save local data. The persistent state is constantly offloaded to external databases or Object Storage (e.g., S3), allowing Kubernetes to "kill" and recreate an infected or failed application node in 1 second, without data loss and without causing interruptions to users.
- **Graceful Degradation:** If Netflix's sophisticated AI recommendation service fails in the cloud, the user interface should not generate a 500 error page. It must degrade the experience "gracefully", automatically serving a trivial generic pre-compiled static list of movies.
- **Retry Logic and Circuit Breaker:** Asynchronous API calls between cloud microservices will fail due to network timeouts or latency. Software patterns are implemented to retry (*Exponential Backoff*) or "open the circuit" (*Circuit Breaker*), completely disabling the problematic call to avoid bombarding and definitively bringing down an already congested microservice.
- **Event-Driven Architecture:** Decoupling via asynchronous queues (e.g. Kafka or RabbitMQ). If a database is temporarily overloaded by an unpredictable traffic spike, incoming writes accumulate patiently in the queue instead of failing, being processed as soon as the database recovers.

5.4 Security

Security in the cloud is based on the concept of **Trust** between customer and provider, built through visibility and control. The goal is to guarantee the **CIA** pillars (Confidentiality, Integrity, Availability).

5.4.1 Classic Threats and Attacks

Historically, datacenters had to defend against attacks aimed at the perimeter (Firewall). Classic threats include:

- **Spoofing:** Forging identity (e.g., IP spoofing) to bypass ACLs.
- **Man-in-the-Middle (MitM) and Sniffing:** Unauthorized interception of traffic in transit.
- **Denial of Service (DDoS):** Saturation of resources (computational or bandwidth) to bring down service availability.
- **Phishing / Social Engineering:** Compromising the human factor to steal valid credentials.

5.4.2 Identity and Policy Management (AAA and RBAC)

Beyond perimeter defense, the vital foundation of cloud security in multi-tenant environments is the correct management of access and privileges:

- **AAA Model:** Represents the fundamental security framework.
 - *Authentication (Who are you?):* Rigorous identity verification (e.g., via passwords, certificates or Multi-Factor Authentication).
 - *Authorization (What can you do?):* Evaluation and granting of the user's exact permissions on individual cloud resources.
 - *Accounting (What did you do?):* Tracking and immutable logging of every action (Audit Trail) for post-incident forensic analysis and billing purposes.
- **RBAC (Role-Based Access Control):** In the cloud, permissions are almost never assigned to the individual user directly (it would be unmanageable). Logical **Roles** are created (e.g. "Storage Admin", "Auditor"). The user assumes the role or is associated with it. This system avoids the uncontrolled proliferation of privileges and ensures compliance with the *Least Privilege* security principle (always and only provide the minimum privilege necessary to perform a task).

5.4.3 Defense-in-Depth and Zero Trust Approach

Since classic attacks have proven that a single secure perimeter does not exist and that threats also come from within (*Insider Threat*), the classic model (based on a single external DMZ) is today replaced by **Zero Trust Architecture (ZTA)**. The principle is: "Never trust, always verify", relying on strong authentication and dynamic *Micro-segmentation* before accessing every single workload. The foundation of the security architecture is the **TCB (Trusted Computing Base)**, the set of critical components (e.g., hypervisor, BMC firmware), which if compromised inevitably bring down the entire chain of trust.

5.4.4 Risk Management (GRC)

Encompassed in the **GRC** (Governance, Risk management, Compliance) framework:

- No system is 100% inviolable. Risk Assessment evaluates the probability of an incident against the cost of the countermeasure.
- Compliance ensures adherence to regulations such as GDPR, NIS2, etc.

Chapter 6

Operations and Final Considerations

6.1 Service Operations Management

This phase closes the datacenter lifecycle by addressing the procedural and operational part: everything that happens *after* the infrastructure is up and running. Keeping a cloud running means ensuring compliance with **SLAs (Service Level Agreements)**, identifying failures, planning resources, and tracking every change. The main activities are grouped into:

- **Monitoring:** Continuously observing the state of the infrastructure and identifying anomalies.
- **Configuration and Change Management:** Tracking inventory and governing every change to the system.
- **Capacity Management:** Understanding how many resources will be needed in the coming months to procure them before they run out.
- **Incident and Problem Management:** Responding to unplanned events and preventing their recurrence.

6.1.1 Monitoring and Predictive Failure

In a modern infrastructure, there are millions of counters (network, hypervisor, BMC). Since every "clean" system generates thousands of warnings in the logs, the real job of monitoring is separating actual problems from background noise using historical data. We distinguish between:

- **Configuration monitoring:** Detecting unauthorized changes or policy violations.
- **Availability monitoring:** Verifying the reachability of services and the status of individual hardware components (e.g., disks).
- **Security monitoring:** Fundamental in public clouds, it monitors logical and physical access (e.g., biometric sensors at doors).

An advanced feature is **Predictive Failure Analysis**: BMCs integrate statistical models and, by analyzing a disk's patterns, issue a warning *before* it breaks. This allows removing it from the pool, replacing it, and reinserting it without any impact on the user.

The redundancy clock and cascading effects

When a component fails, the "clock starts ticking": redundancy is reduced and replacement must be swift. A failure can also generate a **cascading effect**: the load shifted to other healthy nodes overloads them, causing them to crash in turn.

6.1.2 Capacity Monitoring and Management

Capacity Management thinks like an insurance company: statistical estimates are made, and the bet is that not everyone will use resources to the maximum concurrently. This allows the practice of **Overbooking** (or CPU and network *over-commitment*). Some operational techniques include:

- **Dynamic scheduling**: Automatic load balancing among the vCPUs of physical servers.
- **Storage Tiering**: Moving the disks of powered-off VMs to slow, inexpensive storage. Upon power-on, the VM is started and the storage undergoes a background live migration to a fast tier, totally transparent to the user.

6.2 Change and Failure Management

6.2.1 Change Management

It is the process aimed at recording every single change to a system's configuration in the **CMDB (Configuration Management Database)**. When something breaks, the operations team's first question is always "who touched something?". Any structural change (e.g. DB update) requires a strict process: creating a snapshot (guaranteeing reversibility in case of rollback), updating, verifying and, only if successful, deleting the snapshot so as not to burden future I/O performance due to the chain of differential files.

6.2.2 Incident vs. Problem Management

The ITIL distinction between incident and problem is vital for operations:

Incident vs Problem

An **Incident** is an unplanned event that degrades a service (*reactive* approach: resolving the symptom in the shortest possible time). A **Problem** is the root cause of one or more incidents (*proactive* approach: eliminating the cause to prevent it from happening again).

Practical example: a disk breaks and a storage pool runs out (incident). Changing the disk resolves the incident. *Problem management* investigates and discovers that the pool was already too saturated, so the long-term solution is to expand the cluster's overall capacity to prevent future failures.

6.3 Final Considerations on the Datacenter

The evolution of IT has followed a path starting from physical foundations (energy, dissipation) up to total abstraction (cloud). Some summary key concepts:

- **The principle of balance:** An efficient system is not the one with the best absolute component, but the balanced one. A lightning-fast CPU with a mechanical disk storage will result in a slow system. Datacenter design is inherently an exercise in avoiding and managing bottlenecks.
- **Workload divergence:** While hardware was once "general-purpose", today workloads diverge. Artificial Intelligence requires enormous amounts of memory, densely packed GPUs, and ultra-low network latencies, leading to thermal power concentrations (up to gigawatts in single buildings) and extreme direct-to-chip liquid cooling solutions unthinkable a decade ago.
- **Architecting for the load:** Faced with massive data flows (e.g., CERN experiments that produce 10 TB/s but discard 90% after initial processing), the engineer must size frontend and backend asymmetrically, choosing the infrastructure (SAN vs Object/NAS) based on the actual access patterns, acceptable latency, and required parallelism.